S1.02 — Exploration algorithmique d'un problème

Compression avec le codage de Huffman

Clément Pagès
clement.pages@u-pec.fr

Université Paris-Est Créteil
BUT Informatique 1re année

2025-2026

Licence CC-BY-SA

Objectifs

- Pratique de la programmation orientée objet.
- Manipulation de données structurées pour résoudre un problème algorithmique.
- Utilisation d'outils mathématiques (graphes, arbres) en développement informatique.

Licence CC-BY-SA

Sommaire

1. Présentation du sujet
2. Le codage de Huffman
3. Considérations techniques et modalités

Licence CC-BY-SA

Compétences et apprentissages critiques

Compétence 2 Proposer des applications informatiques optimisées en fonction de critères spécifiques : [...] consommation de ressources.

Licence CC-BY-SA

Compétences et apprentissages critiques

Compétence 2 Proposer des applications informatiques optimisées en fonction de critères spécifiques : [...] consommation de ressources.

Compétence 5 [...] Organiser et piloter un projet informatique avec des méthodes classiques ou agiles.

Licence CC-BY-SA

Compétences et apprentissages critiques

Compétence 2 Proposer des applications informatiques optimisées en fonction de critères spécifiques : [...] consommation de ressources.

Compétence 5 [...] Organiser et piloter un projet informatique avec des méthodes classiques ou agiles.

Apprentissage critique 1 Comparer des algorithmes pour des problèmes classiques.

Licence CC-BY-SA

Compétences et apprentissages critiques

Compétence 2 Proposer des applications informatiques optimisées en fonction de critères spécifiques : [...] consommation de ressources.

Compétence 5 [...] Organiser et piloter un projet informatique avec des méthodes classiques ou agiles.

Apprentissage critique 1 Comparer des algorithmes pour des problèmes classiques.

Apprentissage critique 2 Formaliser et mettre en œuvre des outils mathématiques pour l'informatique.

Licence CC-BY-SA

Compétences et apprentissages critiques

Compétence 2 Proposer des applications informatiques optimisées en fonction de critères spécifiques : [...] consommation de ressources.

Compétence 5 [...] Organiser et piloter un projet informatique avec des méthodes classiques ou agiles.

Apprentissage critique 1 Comparer des algorithmes pour des problèmes classiques.

Apprentissage critique 2 Formaliser et mettre en œuvre des outils mathématiques pour l'informatique.

Déroulement 12 h de séances encadrées + 15 heures d'autonomie + travail personnel.

Licence CC-BY-SA

Sujet, problématique

Algorithmes de compression

- Thématique : algorithmes de compression.
- Objectif : réduire la taille d'un ensemble de données (images, sons, vidéos, etc.).

Licence CC-BY-SA

Sujet, problématique

Algorithmes de compression

- Thématique : algorithmes de compression.
- Objectif : réduire la taille d'un ensemble de données (images, sons, vidéos, etc.).
- Deux grandes familles d'algorithmes de compression
  - Compression avec perte : on dégrade volontairement les données en entrée pour gagner de la place. (JPEG, MP3, MP4, etc.)

Licence CC-BY-SA

Sujet, problématique

Algorithmes de compression

- Thématique : algorithmes de compression.
- Objectif : réduire la taille d'un ensemble de données (images, sons, vidéos, etc.).
- Deux grandes familles d'algorithmes de compression
  - Compression avec perte : on dégrade volontairement les données en entrée pour gagner de la place. (JPEG, MP3, MP4, etc.) La donnée d'origine ne peut pas être reconstituée.

Licence CC-BY-SA

Sujet, problématique

Algorithmes de compression

- Thématique : algorithmes de compression.
- Objectif : réduire la taille d'un ensemble de données (images, sons, vidéos, etc.).
- Deux grandes familles d'algorithmes de compression
  - Compression avec perte : on dégrade volontairement les données en entrée pour gagner de la place. (JPEG, MP3, MP4, etc.) La donnée d'origine ne peut pas être reconstituée.
  - Compression sans perte : on trouve une manière plus efficace de représenter l'information. (GZ, ZIP, Huffman, etc.).

Licence CC-BY-SA

Sujet, problématique

Algorithmes de compression

- Thématique : algorithmes de compression.
- Objectif : réduire la taille d'un ensemble de données (images, sons, vidéos, etc.).
- Deux grandes familles d'algorithmes de compression
  - Compression avec perte : on dégrade volontairement les données en entrée pour gagner de la place. (JPEG, MP3, MP4, etc.) La donnée d'origine ne peut pas être reconstituée.
  - Compression sans perte : on trouve une manière plus efficace de représenter l'information. (GZ, ZIP, Huffman, etc.).
- On s'intéresse ici à un algorithme sans perte pour la compression de textes.

Licence CC-BY-SA

Sujet, problématique

Algorithmes de compression

- Thématique : algorithmes de compression.
- Objectif : réduire la taille d'un ensemble de données (images, sons, vidéos, etc.).
- Deux grandes familles d'algorithmes de compression
  - Compression avec perte : on dégrade volontairement les données en entrée pour gagner de la place. (JPEG, MP3, MP4, etc.) La donnée d'origine ne peut pas être reconstituée.
  - Compression sans perte : on trouve une manière plus efficace de représenter l'information. (GZ, ZIP, Huffman, etc.).
- On s'intéresse ici à un algorithme sans perte pour la compression de textes.
- L'algorithme étudié s'appelle le codage de Huffman.

Licence CC-BY-SA

Sujet, problématique

L'algorithme de Huffman

- Utilisé dans des contextes très divers
  - Images : PNG.
  - Surcouche après une compression avec perte.
  - Textes

Licence CC-BY-SA

Sujet, problématique

L'algorithme de Huffman

- Utilisé dans des contextes très divers
  - Images : PNG.
  - Surcouche après une compression avec perte.
  - Textes
- On s'intéresse au codage de Huffman sur des textes.

Licence CC-BY-SA

Sujet, problématique

L'algorithme de Huffman

- Utilisé dans des contextes très divers
  - Images : PNG.
  - Surcouche après une compression avec perte.
  - Textes
- On s'intéresse au codage de Huffman sur des textes.
- Données en entrée de l'algorithme :
  - un texte encodé en ASCII.
  - 1 caractère occupe 1 octet.

Licence CC-BY-SA

Sujet, problématique

L'algorithme de Huffman

- Utilisé dans des contextes très divers
  - Images : PNG.
  - Surcouche après une compression avec perte.
  - Textes
- On s'intéresse au codage de Huffman sur des textes.
- Données en entrée de l'algorithme :
  - un texte encodé en ASCII.
  - 1 caractère occupe 1 octet.
- Données en sortie :
  - le texte compressé avec son nouvel encodage.
  - la table de conversion ASCII/nouvel encodage.

Licence CC-BY-SA

Travail demandé

- Implémenter en Python le codage de Huffman.
- Programme en console.
- L'utiliser avec divers textes en entrée.
- Comparer la taille des textes en entrée avec la taille après compression.

Licence CC-BY-SA

Sommaire

1. Présentation du sujet
2. Le codage de Huffman
   - Principe de l'algorithme
   - Implémentation des structures d'arbres
3. Considérations techniques et modalités

Licence CC-BY-SA

Présentation générale

- Entrée de l'algorithme : une chaine de caractères `s` encodée en ASCII.
- Principe : plus un caractère est fréquent, plus il est encodé sur un petit nombre de bits.

Licence CC-BY-SA

Présentation générale

- Entrée de l'algorithme : une chaine de caractères `s` encodée en ASCII.
- Principe : plus un caractère est fréquent, plus il est encodé sur un petit nombre de bits.
- Algorithme en quatre étapes :

Licence CC-BY-SA

Présentation générale

- Entrée de l'algorithme : une chaine de caractères `s` encodée en ASCII.
- Principe : plus un caractère est fréquent, plus il est encodé sur un petit nombre de bits.
- Algorithme en quatre étapes :
  1. Calculer l'effectif de chaque caractère présent dans `s`.
  2. Construire l'arbre de Huffman associé à `s`.
  3. Calculer, pour chaque caractère, son nouvel encodage.
  4. Encoder `s` avec le nouvel encodage.

Licence CC-BY-SA

Présentation générale

- Entrée de l'algorithme : une chaine de caractères `s` encodée en ASCII.
- Principe : plus un caractère est fréquent, plus il est encodé sur un petit nombre de bits.
- Algorithme en quatre étapes :
  1. Calculer l'effectif de chaque caractère présent dans `s`.
  2. Construire l'arbre de Huffman associé à `s`.
  3. Calculer, pour chaque caractère, son nouvel encodage.
  4. Encoder `s` avec le nouvel encodage.

Notations

- Alphabet $\Sigma_{\mathtt{s}}$: ensemble des caractères présents dans `s`.
- Pour $\mathtt{c} \in \Sigma_{\mathtt{s}}$, on note $e_{\mathtt{c}}$ son effectif dans `s`.
- On note $\mathtt{T}_{\mathtt{s}}$ un arbre de Huffman associé à `s`.

Licence CC-BY-SA

Étape 1 — Effectif de chaque caractère

- Calculer $\Sigma_{\mathtt{s}}$ et calculer les effectifs.
- Les ordonner par effectif décroissant.
- Former la liste L des couples $(\mathtt{c}, e_{\mathtt{c}})$.

Exemple

```
s = "attention a la marche en descendant du train"
```


Licence CC-BY-SA

Étape 1 — Effectif de chaque caractère

- Calculer $\Sigma_{\mathtt{s}}$ et calculer les effectifs.
- Les ordonner par effectif décroissant.
- Former la liste L des couples $(\mathtt{c}, e_{\mathtt{c}})$.

Exemple

`s = "attention a la marche en descendant du train"`

$\Sigma_{\mathtt{s}}$	' '	'a'	'n'	'e'	't'	'd'	'c'	'i'
Effectif	7	6	6	5	5	3	2	2

$\Sigma_{\mathtt{s}}$	'r'	'h'	'l'	'm'	'o'	's'	'u'
Effectif	2	1	1	1	1	1	1

Licence CC-BY-SA

Étape 1 — Effectif de chaque caractère

- Calculer $\Sigma_{\mathtt{s}}$ et calculer les effectifs.
- Les ordonner par effectif décroissant.
- Former la liste L des couples $(\mathtt{c}, e_{\mathtt{c}})$.

Exemple

`s = "attention a la marche en descendant du train"`

$\Sigma_{\mathtt{s}}$	' '	'a'	'n'	'e'	't'	'd'	'c'	'i'
Effectif	7	6	6	5	5	3	2	2

$\Sigma_{\mathtt{s}}$	'r'	'h'	'l'	'm'	'o'	's'	'u'
Effectif	2	1	1	1	1	1	1

```
L = [(' ', 7), ('a', 6), ('n', 6), ('t', 5), ('e',
5), ('d', 3), ('c', 2), ('i', 2), ('r', 2), ('h', 1),
('l', 1), ('m', 1), ('o', 1), ('s', 1), ('u', 1)]
```


Licence CC-BY-SA

Étape 2 — Construction de l'arbre de Huffman

Définition

L'arbre de Huffman $\mathtt{T_s}$ associé à $\mathtt{s}$ est un arbre binaire valué tel que :

Licence CC-BY-SA

Étape 2 — Construction de l'arbre de Huffman

Définition

L'arbre de Huffman T_s associé à s est un arbre binaire valué tel que :

- les feuilles sont les éléments de L triés par effectif décroissant.

Licence CC-BY-SA

Étape 2 — Construction de l'arbre de Huffman

Définition

L'arbre de Huffman $\mathtt{T_s}$ associé à $\mathtt{s}$ est un arbre binaire valué tel que :

- les feuilles sont les éléments de $\mathtt{L}$ triés par effectif décroissant.
- pour tout nœud $\mathtt{N} = (\mathtt{s}', e_{\mathtt{s}'})$ admettant un sous-nœud gauche $\mathtt{N}_g = (\mathtt{s}'_g, e_{\mathtt{s}'_g})$ et un sous-nœud droit $\mathtt{N}_d = (\mathtt{s}'_d, e_{\mathtt{s}'_d})$, on a :

$$\mathtt{s}' = \mathtt{s}'_g + \mathtt{s}'_d \quad \text{et} \quad e_{\mathtt{s}'} = e_{\mathtt{s}'_g} + e_{\mathtt{d}'}$$

Licence CC-BY-SA

Étape 2 — Construction de l'arbre de Huffman

Définition

L'arbre de Huffman $\mathtt{T_s}$ associé à $\mathtt{s}$ est un arbre binaire valué tel que :

- les feuilles sont les éléments de $\mathtt{L}$ triés par effectif décroissant.
- pour tout nœud $\mathtt{N} = (\mathtt{s}', e_{\mathtt{s}'})$ admettant un sous-nœud gauche $\mathtt{N}_g = (\mathtt{s}'_g, e_{\mathtt{s}'_g})$ et un sous-nœud droit $\mathtt{N}_d = (\mathtt{s}'_d, e_{\mathtt{s}'_d})$, on a :

$$\mathtt{s}' = \mathtt{s}'_g + \mathtt{s}'_d \quad \text{et} \quad e_{\mathtt{s}'} = e_{\mathtt{s}'_g} + e_{\mathtt{d}'}$$

- les branches gauches ont la valeur 0 et les branches droites ont la valeur 1.

Licence CC-BY-SA

Étape 2 — Construction de l'arbre de Huffman

Définition

L'arbre de Huffman $\mathtt{T_s}$ associé à $\mathtt{s}$ est un arbre binaire valué tel que :

- les feuilles sont les éléments de $\mathtt{L}$ triés par effectif décroissant.
- pour tout nœud $\mathtt{N} = (\mathtt{s}', e_{\mathtt{s}'})$ admettant un sous-nœud gauche $\mathtt{N}_g = (\mathtt{s}'_g, e_{\mathtt{s}'_g})$ et un sous-nœud droit $\mathtt{N}_d = (\mathtt{s}'_d, e_{\mathtt{s}'_d})$, on a :

$$\mathtt{s}' = \mathtt{s}'_g + \mathtt{s}'_d \quad \text{et} \quad e_{\mathtt{s}'} = e_{\mathtt{s}'_g} + e_{\mathtt{d}'}$$

- les branches gauches ont la valeur 0 et les branches droites ont la valeur 1.

Licence CC-BY-SA

Étape 2 — Construction de l'arbre de Huffman

Définition

L'arbre de Huffman $\mathtt{T_s}$ associé à $\mathtt{s}$ est un arbre binaire valué tel que :

- les feuilles sont les éléments de $\mathtt{L}$ triés par effectif décroissant.
- pour tout nœud $\mathtt{N} = (\mathtt{s}', e_{\mathtt{s}'})$ admettant un sous-nœud gauche $\mathtt{N}_g = (\mathtt{s}'_g, e_{\mathtt{s}'_g})$ et un sous-nœud droit $\mathtt{N}_d = (\mathtt{s}'_d, e_{\mathtt{s}'_d})$, on a :

$$\mathtt{s}' = \mathtt{s}'_g + \mathtt{s}'_d \quad \text{et} \quad e_{\mathtt{s}'} = e_{\mathtt{s}'_g} + e_{\mathtt{d}'}$$

- les branches gauches ont la valeur 0 et les branches droites ont la valeur 1.

Exemple

Arbre de Huffman associé à `s = "attention a la marche en descendant du train"`

Licence CC-BY-SA

Étape 2 — Construction de l'arbre de Huffman

Définition

L'arbre de Huffman $\mathtt{T_s}$ associé à $\mathtt{s}$ est un arbre binaire valué tel que :

- les feuilles sont les éléments de $\mathtt{L}$ triés par effectif décroissant.
- pour tout nœud $\mathtt{N} = (\mathtt{s}', e_{\mathtt{s}'})$ admettant un sous-nœud gauche $\mathtt{N}_g = (\mathtt{s}'_g, e_{\mathtt{s}'_g})$ et un sous-nœud droit $\mathtt{N}_d = (\mathtt{s}'_d, e_{\mathtt{s}'_d})$, on a :

$$\mathtt{s}' = \mathtt{s}'_g + \mathtt{s}'_d \quad \text{et} \quad e_{\mathtt{s}'} = e_{\mathtt{s}'_g} + e_{\mathtt{d}'}$$

- les branches gauches ont la valeur 0 et les branches droites ont la valeur 1.

Exemple

Arbre de Huffman associé à `s = "attention a la marche en descendant du train"`

Fait en amphi : il faut venir en cours.

Licence CC-BY-SA

Étape 3 — Calcul du nouvel encodage

- Pour chaque caractère, partir de la racine de l'arbre.

Licence CC-BY-SA

Étape 3 — Calcul du nouvel encodage

- Pour chaque caractère, partir de la racine de l'arbre.
- Suivre le chemin partant de la racine, menant à la feuille.

Licence CC-BY-SA

Étape 3 — Calcul du nouvel encodage

- Pour chaque caractère, partir de la racine de l'arbre.
- Suivre le chemin partant de la racine, menant à la feuille.
- Si l'on part à gauche, on encode un 0. Si l'on part à droite, on encode un 1.

Licence CC-BY-SA

Étape 3 — Calcul du nouvel encodage

- Pour chaque caractère, partir de la racine de l'arbre.
- Suivre le chemin partant de la racine, menant à la feuille.
- Si l'on part à gauche, on encode un 0. Si l'on part à droite, on encode un 1.

Licence CC-BY-SA

Étape 3 — Calcul du nouvel encodage

- Pour chaque caractère, partir de la racine de l'arbre.
- Suivre le chemin partant de la racine, menant à la feuille.
- Si l'on part à gauche, on encode un 0. Si l'on part à droite, on encode un 1.

Exemple

```
s = "attention a la marche en descendant du train"
```


Licence CC-BY-SA

Étape 3 — Calcul du nouvel encodage

- Pour chaque caractère, partir de la racine de l'arbre.
- Suivre le chemin partant de la racine, menant à la feuille.
- Si l'on part à gauche, on encode un 0. Si l'on part à droite, on encode un 1.

Exemple

`s = "attention a la marche en descendant du train"`

Σ_s	' '	'a'	'n'	'e'	't'
Eff.	7	6	6	5	5
Codage	101	110	111	000	001

Σ_s	'd'	'c'	'i'	'r'	'h'
Eff.	3	2	2	2	1
Codage	1001	10000	10001	0110	01010

Σ_s	'l'	'm'	'o'	'u'	's'
Eff.	1	1	1	1	1
Codage	01011	01000	01001	01111	01110

Licence CC-BY-SA

Structure d'arbre binaire en Python

- Gestion des arbres binaires à implémenter en Python.

Licence CC-BY-SA

Structure d'arbre binaire en Python

- Gestion des arbres binaires à implémenter en Python.
- Structure récursive : valeur du nœud, sous-arbre gauche, sous-arbre droit.

Licence CC-BY-SA

Structure d'arbre binaire en Python

- Gestion des arbres binaires à implémenter en Python.
- Structure récursive : valeur du nœud, sous-arbre gauche, sous-arbre droit.
- On va plutôt créer une classe `NoeudBinaire`
  - Quels sont ses attributs ?

Licence CC-BY-SA

Structure d'arbre binaire en Python

- Gestion des arbres binaires à implémenter en Python.
- Structure récursive : valeur du nœud, sous-arbre gauche, sous-arbre droit.
- On va plutôt créer une classe `NoeudBinaire`
  - Quels sont ses attributs ?
- Opérations élémentaires à implémenter :
  - Getters, setters.
  - Test si un arbre est vide, est une feuille, admet un sous-arbre gauche/un sous-arbre droit.
  - Parcours, affichage, ajout d'un nouveau nœud.

Licence CC-BY-SA

Structure d'arbre de Huffman

- Arbre de Huffman : arbre binaire valué.

Licence CC-BY-SA

Structure d'arbre de Huffman

- Arbre de Huffman : arbre binaire valué.
- Une approche possible : créer une classe `NoeudHuffman` qui hérite de la classe `NoeudBinaire`.

Licence CC-BY-SA

Structure d'arbre de Huffman

- Arbre de Huffman : arbre binaire valué.
- Une approche possible : créer une classe `NoeudHuffman` qui hérite de la classe `NoeudBinaire`.
- Les valeurs des nœuds sont des tuples $(\mathtt{s}', e_{\mathtt{s}'})$ où $\mathtt{s}'$ est une liste d'éléments de $\Sigma_{\mathtt{s}}$.

Licence CC-BY-SA

Sommaire

1. Présentation du sujet
2. Le codage de Huffman
3. Considérations techniques et modalités
   - Livrables et exécution
   - Déroulé, modalités d'évaluation

Licence CC-BY-SA

Livrables et mise en œuvre

- Programmation orientée objet en Python.

Licence CC-BY-SA

Livrables et mise en œuvre

- Programmation orientée objet en Python.
- Livrables :
  - codes sources.
  - dépôt Git (local ou distant).
  - taux de compression obtenus (dans des fichiers csv ou txt).

Licence CC-BY-SA

Livrables et mise en œuvre

- Programmation orientée objet en Python.
- Livrables :
  - codes sources.
  - dépôt Git (local ou distant).
  - taux de compression obtenus (dans des fichiers csv ou txt).
- Le code doit pouvoir être exécuté depuis n'importe quelle machine depuis le terminal, avec un fichier `main.py` comme programme principal.

Licence CC-BY-SA

Livrables et mise en œuvre

- Programmation orientée objet en Python.
- Livrables :
  - codes sources.
  - dépôt Git (local ou distant).
  - taux de compression obtenus (dans des fichiers csv ou txt).
- Le code doit pouvoir être exécuté depuis n'importe quelle machine depuis le terminal, avec un fichier `main.py` comme programme principal.
- Afficher (au moins) le code de Huffman et le taux de compression obtenu.

Licence CC-BY-SA

Livrables et mise en œuvre

- Programmation orientée objet en Python.
- Livrables :
  - codes sources.
  - dépôt Git (local ou distant).
  - taux de compression obtenus (dans des fichiers csv ou txt).
- Le code doit pouvoir être exécuté depuis n'importe quelle machine depuis le terminal, avec un fichier `main.py` comme programme principal.
- Afficher (au moins) le code de Huffman et le taux de compression obtenu.

Licence CC-BY-SA

Modules externes

Utilisation de modules externes interdite

- Pour cette SAÉ, il est interdit d'utiliser des modules Python externes.
- L'implémentation des structures de données fait partie du travail.

Licence CC-BY-SA

Modules externes

Utilisation de modules externes interdite

- Pour cette SAÉ, il est interdit d'utiliser des modules Python externes.
- L'implémentation des structures de données fait partie du travail.

- Trois exceptions
  - `sys` : récupérer des paramètres depuis le terminal.
  - `os` : manipuler les dossiers et fichier.
  - `unidecode` : gestion de l'encodage ASCII

Licence CC-BY-SA

Exécution du code

- Le code lit le contenu d'un dossier passé en paramètre depuis le terminal.
- Les fichiers `*.txt` de ce répertoire sont compressés.
- La sauvegarde du résultat sous forme compressée n'est pas demandée.

Licence CC-BY-SA

Exécution du code

- Le code lit le contenu d'un dossier passé en paramètre depuis le terminal.
- Les fichiers `*.txt` de ce répertoire sont compressés.
- La sauvegarde du résultat sous forme compressée n'est pas demandée.

Terminal

```
$ ls
input/ main.py # Autre fichiers *.py…
$ ls input/ # Fichiers à compresser
aLaRechercheDuTempsPerdu.txt  leSavetierEtLeFinancier.txt
lesMiserables.txt
$ # Exécution du script main.py, avec en paramètre 'input/'
$ python3 main.py input/
```


Licence CC-BY-SA

Modalités pratiques

- Planning prévisionnel
  - Durée : 4 semaines.
  - Fin : semaine du 13/04.
  - 7 équipes par groupe de TD, ayant des effectifs équilibrés.

Licence CC-BY-SA

Modalités pratiques

- Planning prévisionnel
  - Durée : 4 semaines.
  - Fin : semaine du 13/04.
  - 7 équipes par groupe de TD, ayant des effectifs équilibrés.
- Modalités d'évaluation
  - Un devoir écrit : construction à la main d'un arbre de Huffman à partir d'une phrase donnée.
  - Une soutenance de revue de code avec exécution en direct.

Licence CC-BY-SA

Dépôt intermédiaire de code

Gestion des arbres binaires et de Huffman

- Pour le 05/04 — Implémentation des classes gérant les arbres binaires et les arbres de Huffman.

Licence CC-BY-SA

Dépôt intermédiaire de code

Gestion des arbres binaires et de Huffman

- Pour le 05/04 — Implémentation des classes gérant les arbres binaires et les arbres de Huffman.
- À déposer sur EPREL.

Licence CC-BY-SA

Dépôt intermédiaire de code

Gestion des arbres binaires et de Huffman

- Pour le 05/04 — Implémentation des classes gérant les arbres binaires et les arbres de Huffman.
- À déposer sur EPREL.
- Une correction sera fournie pour ne pas rester bloquer.
- Son utilisation est facultative si vous n'avez pas réussi à implémenter les fonctionnalités demandées...

Licence CC-BY-SA

Dépôt intermédiaire de code

Gestion des arbres binaires et de Huffman

- Pour le 05/04 — Implémentation des classes gérant les arbres binaires et les arbres de Huffman.
- À déposer sur EPREL.
- Une correction sera fournie pour ne pas rester bloquer.
- Son utilisation est facultative si vous n'avez pas réussi à implémenter les fonctionnalités demandées...
- ... mais elle est pénalisée.

Licence CC-BY-SA

Dépôt final

- Pour le 19/04
- Code source, dépôt Git ou lien vers le dépôt distant.
- Résultats de compression obtenus pour chaque texte en entrée.

Licence CC-BY-SA

Soutenance

- Durée : 10 minutes démonstration comprise.
- Rapide introduction : problématique, organisation de l'équipe.
- Présentation du code et de sa structure.
- Exécution en directe.
  - Avec les données fournies.
  - Ou avec d'autres plus petites, pour avoir des exemples simples.
  - Affichage des résultats et des taux de compression.

Licence CC-BY-SA